Assumptions

Where the instructions left something open, I picked the simplest reasonable option. This file lists those choices so they can be checked against what was actually intended.

How to read the supplied planning notes

The three-page `Dog_Group_Notes.pdf` is handwritten planning material. It is treated as design context, not as a set of executable specifications or instructions that override the current request. The sketches appear to raise ideas about (1) dog-game states and joint-action Q values, (2) continuous best responses, gradients and angle-radius actions, and (3) a pursuit/chasing game with moving targets. The handwriting leaves important rules unspecified. The repository implements the separately documented dog and walking games; it does not implement a chase game from the third sketch. In the current request, the user resolved the original dog's angle-radius action as an absolute point from the square center. The existing walking game's move-from-current-position interpretation remains distinct. The Q/DQN material was explicitly excluded from this implementation.

The game

- The domain is the unit square, and the houses are fixed points.
- The default transition is `dog = w * red + (1 - w) * blue`. It ignores the dog's previous position.
- The reward is the negative squared distance from the dog to a player's own house.
- The discount is 0.9 (0.95 in `DogGameEnv` by default). Neither value was specified.
- The dog starts at `w * house_red + (1 - w) * house_blue`.
- `make_inertial_transition` reads `w0 * dog` as the dog's own previous position and forces `w0 + w1 + w2 = 1`. The Battling Influencers paper (arXiv 2502.01127) instead defines `x0` as a fixed background point, with weights that need not be normalized. My reading may be wrong.
- The live site updates players one after the other (red responds, then blue responds to the new red), not simultaneously. Its default house positions and its 31x31 exploitability grid are my choices.

Policy gradient

- The policy is Gaussian. The mean goes through a sigmoid, and the std is a learned value that doesn't depend on the state.
- Actions are clipped to the square before the log-probability is taken, which makes the gradient slightly biased.
- REINFORCE uses the mean return within the episode as its baseline. A2C and PPO (`train_ac.py`) use a learned value function of the dog's position, one per player; PPO reuses each episode for 4 epochs with the ratio clipped at 0.2.
- Episodes are 20 steps long, trained with Adam at lr 0.01. Both players update together on one optimizer.
- The state is only the dog's position, and a new policy is trained for each house configuration.

Polar-action policy gradient on the original dog game

- The user clarified that `(theta, radius)` is an absolute action point centered on the square, not a movement from the player's current position. For a square `[low, high]^2`, let `c = (low + high)/2`, `h = (high - low)/2`, and `u` be a normalized radial fraction in `[0, 1]`. The selected point is `a = c + r*(cos(theta), sin(theta))`, with `r = h*u / max(|cos(theta)|, |sin(theta)|)`. This direction-dependent mapping covers the square and returns a feasible action without clipping.
- The policy observes only the current dog position (2 coordinates), as in the original dog game. Houses remain fixed per training run and are not inputs. Separate player networks produce a von Mises angle and Beta radial fraction; their concentration/shape parameters are at least 1, a single-peaked policy-family restriction.
- Likelihoods are computed for latent `(theta, u)`. The action transform is parameter-independent and one-to-one except at the square center, so its Jacobian contributes no actor-parameter gradient and cancels from PPO likelihood ratios. The continuous radial distribution assigns probability zero to the exact center; there `atan2(0, 0)` supplies a conventional angle.
- This differs from the existing walking game, where angle and step length move each player's current position. No DQN, joint-action Q network, or learned best-response surface is part of this PG comparison.

Polar-action comparison protocol

The frozen plan is in `docs/POLAR_ACTION_PROTOCOL.md`. It compares the existing clipped Cartesian Gaussian, a bounded Cartesian Beta control, and the polar Beta/von-Mises policy on one primary house configuration, for REINFORCE, A2C and PPO, across ten seeds and 1500 episodes. The primary score accounts for action means and covariance using the exact best response to those estimated moments and is evaluated at the reset state only, not over a full training-state distribution. The 8192-action Monte Carlo estimate, one setup, ten seeds, and legacy threshold are limitations; intervals reflect seed variation but omit Monte Carlo estimation error, and comparisons are exploratory. The clipped-Gaussian baseline retains its existing clipping/log-density behavior, so the comparison is not a pure geometry ablation.

A separate REINFORCE follow-up covers the other four predefined house setups, ten seeds each, and compares only polar Beta/von-Mises with Cartesian Beta. Its protocol was specified after the primary study began, so it is a separate exploratory analysis rather than part of the primary pre-run protocol. Its unadjusted intervals also omit Monte Carlo scoring error.

Angle-radius version (`polar.py`, `polar_agent.py`)

- For the existing walking-game variant, `(theta, r)` is a displacement from each player's current position, with `r` capped at `max_step`; players start on their houses. This is a separate modeling choice. It does not change the user-confirmed absolute, square-center action for the new policy-gradient implementation on the original dog game.
- The dog is `w0 * dog + w1 * red + w2 * blue`, where red and blue are the players' positions. Positions are clipped at the edge of the square.
- The angle is sampled from a von Mises and the step fraction from a Beta (both parameters at least 1, so each is single-peaked). I chose these so the angle wraps cleanly and nothing needs clipping.
- The network sees the dog and both players' positions (6 numbers), with one small network per player. Training is REINFORCE with the same mean-return baseline as above, 15 steps per episode.

- The reference answer is the one-shot Nash corners from `nash.py`. Walking there is only checked on the default house setup; other setups were only checked for NaNs.
- The 10-direction deep Nash-Q (`polar_dqn.py`) lets each player walk one of 10 evenly spaced compass directions, always a full `max_step`. One network outputs both players' 10x10 Q-values. The target is `r + discount * Nash value` of the target network's Q at the next state, with a replay buffer and a frozen target network. Each player explores with a random move at a decaying rate, and otherwise samples the Nash strategy of its Q.
- Each stage game is solved exactly. A game whose pivoting cycles is cut off after a fixed number of pivots and retried from another starting label (see below). It used to be cut off by a time limit, which made training depend on how busy the machine was.
- `continuous_br.py` has the three continuous best-response methods from the notes (bisection, finite-difference gradient, quadratic fit). They assume a single peak and are standalone: the deep Nash-Q uses discrete directions and does not call them.
- The simulator's walking mode is not learned. It moves each player toward their best-response pick by at most `max_step`.

Soccer (`soccer*.py`)

- The group's own soccer game wasn't available, so this is my version: a deterministic, two-player grid soccer game with 4 moves, ball possession, and a three-cell goal on a 7x5 board (a smaller 5x3 board with a one-cell goal is also used). Both players move at the same time. The collision and tackle rules are in the `soccer.py` docstring, and I wrote them so the game is identical for both players. They are not the "A10" rules.
- A score ends the game (+1 / -1). The exact solution is discounted with 0.9. Games are cut off at 100 steps and count as ties. The network input has no step counter, because 0.9 to the 100th power is negligible.
- This version has no states that need a mixed strategy, which keeps the comparison simple (fewer mixed-equilibrium states is better). It also means it can't show whether PG finds mixed strategies.
- Policy gradient never solves a game. The exact solution is used only to judge the learned policies.
- Training settings (batch of 256 games, lr 0.003, entropy bonus 0.001) are my choices. The entropy and lr were picked by trying four combinations on seed 0 only. Results swung a lot between settings.
- "Exploitability" is how much an exact best response wins against the learned policy. It is harsh: a policy that never loses to the exact equilibrium can still score high, because a best response searches out every state it plays badly.

What the soccer runs showed

Five seeds (0-4) on the 5x3 board, 500 self-play iterations or 200 fictitious-play rounds of 20 iterations each. Player 0, wins/losses/ties per 1000 games, averaged over the seeds. "Worst" is the most games lost to the exact equilibrium by any one seed. Exploitability is the mean, with the range over seeds in brackets.

method	vs random	vs exact equilibrium	worst seed	vs exact best response	exploitability
REINFORCE self-play	941/53/6	0/297/703	500	0/871/129	0.89 (0.61-1.09)
REINFORCE fictitious play	902/89/10	0/161/839	651	0/1000/0	1.00 (0.94-1.03)

method	vs random	vs exact equilibrium	worst seed	vs exact best response	exploitability
A2C self-play	922/73/4	0/500/500	501	0/1000/0	1.04 (0.96-1.10)
A2C fictitious play	910/83/6	0/490/510	1000	0/1000/0	0.99 (0.93-1.02)
PPO self-play	954/17/30	0/100/900	500	1/401/597	0.40 (0.12-0.81)
PPO fictitious play	916/79/5	0/279/721	540	2/998/0	0.96 (0.86-1.11)

- None of them replicates the exact solution. Even the best method is beaten by the exact best response, and no seed reaches exploitability 0.
- PPO self-play is the best on average (exploitability 0.40, and it holds ties against the best response in 597 games), but the seeds vary a lot (0.12 to 0.81).
- The first single-seed table was misleading. It showed REINFORCE self-play drawing all 1000 games against the exact equilibrium. Averaged over five seeds it loses 297, and one seed loses 500.
- Fictitious play did not converge, even at 5x the earlier length. The average policy's exploitability is flat between round 40 and round 200 (REINFORCE 0.99 to 1.00, A2C 0.99 to 0.99, PPO 0.92 to 0.96), which is about where random play sits (1.12). The diagnosis below looked into why, and found two settings that make it improve (they were not used for this table).
- Every method beats random play (90-95% wins).
- Five seeds on one small board is still a small sample. The entropy and lr were tuned on seed 0, which is one of the five.

Why fictitious play didn't converge

REINFORCE on the 5x3 board, 20 iterations of training per round. Exploitability of the average policy (and of the latest policy). The first six variants are 2 seeds and 60 rounds; the last two are 4 seeds and 200 rounds. Details are in `doggame/soccer_fp_diagnosis.py`; raw numbers are in `results/soccer_fp_diagnosis_runs.jsonl`.

variant	average policy	latest policy
exact best responses (no learning), 400 rounds	0.011	-
baseline	0.97	1.03
train against the state-by-state average opponent	0.97	0.96
10x longer best responses (200 iterations x 20 rounds)	1.00	1.03
both of those	0.97	0.96
random starting states	0.95	0.84
random starting states + state-average opponent	0.82	0.41
baseline, 200 rounds	1.00	0.91
random starting states + state-average opponent, 200 rounds	0.58	0.20

- With exact best responses, fictitious play converges (0.87 down to 0.011), so the averaging itself is fine. The problem is in the learned best responses.
- Training each best response ten times longer did not help, and neither did facing the state-by-state average opponent alone. Those two were my first explanations, and they are ruled out as the main cause.

- Only the combination moved it: random starting states plus facing the same average policy that is scored. Over 200 rounds the average policy keeps improving (0.85, 0.73, 0.65, 0.58 at rounds 50, 100, 150, 200) and the latest policy falls from 0.47 to 0.20, while the baseline stays flat at 1.00.
- My next explanation was that policy gradient only improves the states it plays through. I tested it, and it is not supported: of the states an exact best response reaches, 0% had been trained on fewer than 10 times, in either the baseline or the combined setting. That is a crude threshold, so it does not prove coverage is irrelevant, but I don't know why the combination helps.
- It is still not converged (0.58 average at round 200), and I don't know whether it would reach 0. The new settings are options (`mix="state"`, `random_starts=True`), not the defaults, and the seed results above were run without them.

Fictitious play with PPO

PPO with the two settings that helped REINFORCE (random starting states and the state-average opponent), 5x3 board, 20 iterations per round, 400 rounds, 8 seeds (0-7). Exploitability of the average policy and of the latest policy, mean over seeds with the range in brackets. Raw numbers are in `results/soccer_fp_diagnosis_runs.jsonl`.

round	average policy	latest policy	seeds with latest policy under 0.05
50	0.49 (0.19-0.85)	0.28 (0.00-0.79)	4 of 8
100	0.40 (0.09-0.73)	0.12 (0.00-0.49)	5 of 8
150	0.35 (0.06-0.67)	0.10 (0.00-0.49)	6 of 8
200	0.31 (0.04-0.61)	0.10 (0.00-0.50)	6 of 8
250	0.29 (0.03-0.57)	0.12 (0.00-0.49)	6 of 8
300	0.28 (0.03-0.53)	0.13 (0.00-0.49)	5 of 8
350	0.27 (0.02-0.50)	0.12 (0.00-0.49)	6 of 8
400	0.26 (0.02-0.47)	0.20 (0.00-0.60)	5 of 8

- PPO is clearly better than REINFORCE here. At round 200 the average policy is at 0.31 (REINFORCE: 0.58 on 4 seeds) and the latest policy at 0.10 (REINFORCE: 0.20).
- The average policy keeps improving, slowly: 0.49 at round 50 down to 0.26 at round 400, about 0.01 per 50 rounds by the end. It has not converged and no seed reaches 0.
- The seeds differ a lot. Two end with an almost unexploitable average policy (seed 1: 0.03, seed 3: 0.02). One more is at 0.15, and the other five are between 0.30 and 0.47. Seed 7 never gets below about 0.46.
- The latest policy is not stable, so don't read it as a result. It is exactly 0.00 for most seeds most of the time, but seed 0 sat at 0.00 from round 100 to round 350 and then jumped to 0.60 at round 400. Seed 6 jumped from 0.00 to 0.49 at round 250 and stayed there, and seed 2 bounced between 0.79, 0.32, 0.00 and 0.05. A good-looking latest policy can be a lucky snapshot. Fictitious play only promises something about the average policy.
- For every seed, 0% of the states an exact best response reaches were trained on fewer than 10 times, so the state-coverage explanation is still not supported.
- Why the average declines so slowly is tested in the next section.
- This is one small board and one set of settings.

Eight more seeds, 800 rounds, and the early policies

Same PPO settings, seeds 8-15, run to 800 rounds. Exploitability, mean over the 8 seeds with the range in brackets. Raw numbers are in `results/soccer_fp_diagnosis_runs.jsonl`.

round	average policy	latest policy	seeds with latest policy under 0.05
100	0.31 (0.09-0.55)	0.16 (0.00-0.46)	5 of 8
200	0.26 (0.04-0.48)	0.12 (0.00-0.49)	6 of 8
300	0.23 (0.03-0.49)	0.12 (0.00-0.49)	6 of 8
400	0.23 (0.02-0.72)	0.10 (0.00-0.78)	7 of 8
500	0.22 (0.02-0.73)	0.13 (0.00-0.62)	6 of 8
600	0.20 (0.02-0.73)	0.04 (0.00-0.32)	7 of 8
700	0.18 (0.02-0.73)	0.04 (0.00-0.32)	7 of 8
800	0.17 (0.02-0.71)	0.04 (0.00-0.32)	7 of 8

- Over all 16 seeds (0-15) the average policy at round 400 is 0.25 on average (0.02-0.72), and 5 of the 16 are under 0.05.
- The average keeps falling after round 400: 0.23 at round 400, 0.17 at round 800, about 0.015 per 100 rounds. It still has not reached 0.
- Seven of the eight seeds end at 0.22 or lower. Seed 15 is the exception: its average is 0.71 at round 800 and its latest policy is 0.32. Without seed 15 the mean is 0.09.

To test the guess that the average declines slowly because it still contains the weak early policies, each checkpoint holds every snapshot so far, so the same snapshots can be averaged in different ways (`python3 -m doggame.soccer_averaging --rounds 800 soccer_fp_checkpoints/*.pt`). Exploitability, mean over the 8 seeds; raw output is in `results/soccer_averaging_round_400.txt` and `results/soccer_averaging_round_800.txt`.

way of averaging	at round 400	at round 800
all snapshots (the average policy)	0.23	0.17
without the first quarter	0.18	0.13
second half only	0.16	0.12
last quarter only	0.12	0.07
latest snapshot only	0.10	0.04

At round 800, per seed:

seed	all	without first quarter	second half	last quarter	latest
8	0.13	0.04	0.00	0.00	0.00
9	0.22	0.10	0.11	0.00	0.00
10	0.02	0.01	0.01	0.02	0.00
11	0.07	0.03	0.00	0.00	0.00
12	0.05	0.00	0.00	0.00	0.00
13	0.11	0.14	0.18	0.00	0.00
14	0.06	0.08	0.13	0.26	0.00

seed	all	without first quarter	second half	last quarter	latest
15	0.71	0.67	0.54	0.32	0.32

- The guess is partly right. The more of the early history is dropped, the lower the mean exploitability, at both rounds.
- The part from the first quarter alone is small: 0.23 to 0.18 at round 400 and 0.17 to 0.13 at round 800. At round 800, dropping the first quarter helps 6 of the 8 seeds and hurts 2 (seeds 13 and 14).
- It does not explain every seed. In seeds 13 and 14 the average gets worse when the early policies are dropped (seed 14: 0.06 for all snapshots, 0.26 for the last quarter). Seed 15 is poor under every way of averaging (0.71 to 0.32), so something other than the early policies is wrong there. At round 400 seeds 8 and 11 also barely changed.
- "Latest snapshot only" has the lowest mean, but it is one policy, and earlier seeds showed it can jump (seed 0 went from 0.00 to 0.60), so I don't treat it as a result.
- This shows that the early policies add to the exploitability. It does not test the 1-over-the-number-of-rounds explanation of how fast they fade. It is one checkpoint per seed and 8 seeds.

Solvers and checks

- The analytic solver finds one pure Nash equilibrium by iterated best response. When equilibria are not unique it returns an arbitrary one.
- Fictitious play runs 1000 iterations from a random start (seed 0). Lemke-Howson returns the first valid equilibrium it finds. NashPy overflows or cycles on a few games, so each answer is checked and a failed attempt is retried from the next starting label.
- The original dog-game "DQN" is tabular Nash-Q on a single state, with no neural network. The angle-radius game has a real one (below).
- Exploitability is searched on a 101x101 grid, and "converged" means below 0.004 (my threshold).

Evidence

- The dog-game convergence study uses five seeds per cell (75 runs), the other studies five seeds, and the equilibrium-selection study ten per layout. "Converged" means exploitability below 0.004, which is my threshold.
- The "3000 more episodes brought it to 0.004" figure came from an earlier single-seed note. I have not re-run it.
- The studies in `dog_game_studies.py` use my own designs: pre-training is 1500 self-play episodes on another setup before training on the target, inertia runs use `w0` of 0, 0.3, 0.6 and 0.9, and the stage-solver study uses fictitious play with 10 to 3000 iterations against the exact solver. PPO and A2C are compared with REINFORCE on episodes, not gradient steps.
- Timings in the resolution study were taken on a heavily loaded machine.
- The walking-game comparison (study `walking`) uses the same house setup as the other walking runs, 300 episodes, 5 seeds, and scores each checkpoint (every 25 episodes) as the mean of 10 sampled games. "At the corners" means a summed distance from the one-shot Nash corners under 0.1, which is my threshold. A first run scored only at 4 checkpoints gave different per-seed outcomes, because evaluating a policy draws random numbers and so changes the training run, so the paths are sensitive.

- The explanations in the research report for why PG picks one equilibrium over another are hypotheses, not tested results.
- Added exploratory runs in the results file use the original five seeds (0-4); they are not independent confirmation. PPO width comparisons and PPO-with-inertia comparisons use five seeds per setting; no entropy bonus was tested.
- The inertia profile check starts the dog at the weighted house midpoint and holds the opponent at its one-shot Nash action. A coordinate-wise sufficient condition analytically certifies the tested profiles for any horizon and `w0` in [0, 1]; the checked conditions are specific to the five tested setups and do not prove the result for arbitrary houses or weights. Uncertified profiles fall back to a box-constrained convex quadratic optimizer with its analytic gradient and success status checked. A corrected negative control uses an opponent action of (0, 0), fails the certificate and yields a positive red-player gain under the optimizer. This does not solve feedback strategies.
- The live simulator's displayed number remains an unconstrained, one-shot static-grid deviation proxy (31 by 31). It ignores the current dog state, inertia and walking-step constraints; it is not a dynamic-game exploitability measure.
- The literature section is an initial scoped comparison, not a systematic review. The static dog game overlaps the Battling Influencers Game (BIG) structure; novelty is not claimed for the basic static equilibrium result.

Feedback equilibrium and potential tests

- The dynamic program treats the two coordinates as separate 1D games. This is exact here because rewards, dog dynamics and the action box all act coordinate by coordinate; it would not hold for a round action set.
- Each 1D game uses 401 dog positions and 101 actions on [0, 1], discount 0.9, and linear interpolation of values between grid positions. Value iteration stops when values change by less than 1e-10 or after 2000 sweeps.
- At each state the stage game is solved for a pure equilibrium on the action grid by iterated best response, starting the first sweep from red at 1 and blue at 0 and each later sweep from the previous actions; ties go to the lowest action index. Off-grid dog positions use the nearest grid state. States where no pure equilibrium is found within 200 rounds are counted as unresolved (56 states in one setting) and keep the last iterate. I did not compute mixed equilibria there.
- When the stage game has several equilibria (the shared-y setup), the program returns one of them, so a different answer from the one-shot profile there is a selection, not a disagreement. I do not claim the feedback equilibrium is unique.
- Whether holding the one-shot actions everywhere is Markov-perfect is decided by the one-shot deviation principle on a 41 by 41 grid of dog positions, using the closed-form value of constant actions and an exact per-coordinate maximization of the deviation. This is a check on that grid, not a proof for every dog position.
- The Markov-potential test uses one 4-cycle per setting, on setup 1, from the starting dog position. Both players start from their one-shot Nash corners. The alternatives are my own choice: constant actions (0.6, 0.4) and (0.3, 0.7) for "fixed", and red playing 1 - x and blue playing x at dog position x for "reacting". Values are summed over 400 deterministic rounds, where the discounted tail is below 1e-18. A nonzero cycle proves the game is not a Markov potential game; zero cycles for constant policies are only evidence for the open-loop claim, which the one-shot potential also implies.

- The selection-noise study varies only the initial log standard deviation of the policy (-2, -1, -0.5 by default, and 0), with 10 seeds each on the shared-y setup and the `separate` layout. Fisher's exact test compares the counts of the two mirror outcomes; I did not correct for multiple comparisons.

Settled points

- **Which game for PG:** any one where the solver is correct and the comparison is meaningful; fewer mixed-equilibrium states is better.
- **Discounting:** PG can only approximate finite-horizon discounted rewards, so use both (discounted, with a 100-step limit).
- **Fictitious play:** it is how PG solves things. Each player learns a best response to the opponent's earlier policies. There is no explicit game solving in PG, and it is not applied to the exact stage-game matrices.
- **Win rate:** play repeated games and count the wins.
- **Continuous-action methods** (bisection, finite-difference gradient, quadratic fit): those are for DQN. PG handles continuous actions directly, as the angle-radius policy does.

Still open

1. In `w0*dog + w1*x_red + w2*x_blue`, are `x_red` and `x_blue` the players' positions? Is `w0*dog` the dog's current position or the paper's fixed `x0`? Must the weights sum to 1?
2. Should the soccer comparison use the group's own game (7x5, random move order, or the "A10" rules) instead of mine?
3. How should the angle's wraparound and the board edge be handled in other future variants? The new main-game implementation wraps angles and maps radii to the square boundary; the original note does not specify another convention.